

Artificial Intelligence for Science



Lesson 13: Recurrent Neural Network (循环神经网络)

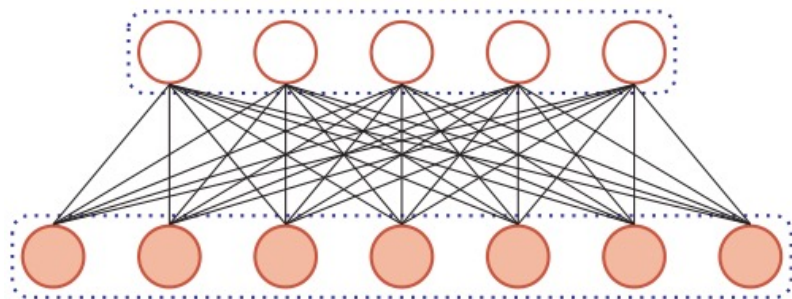
CHEN Kejie 陈克杰 (chenkj@sustech.edu.cn)

13-May-26

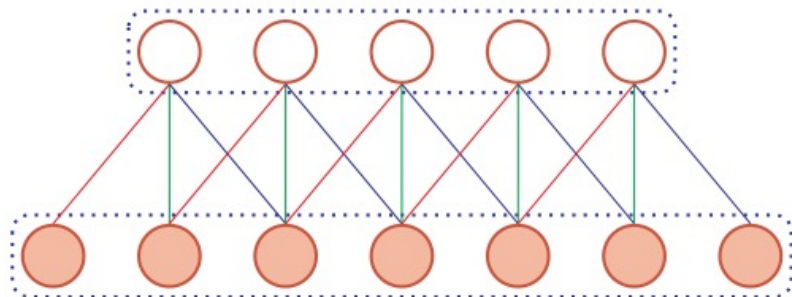


前馈网络 (Forward Feed Neural Network)

- ▶ 连接存在层与层之间，每层的节点之间是无连接的（无循环）。
- ▶ 输入和输出的维数都是固定的，不能任意改变。无法处理变长的序列数据。



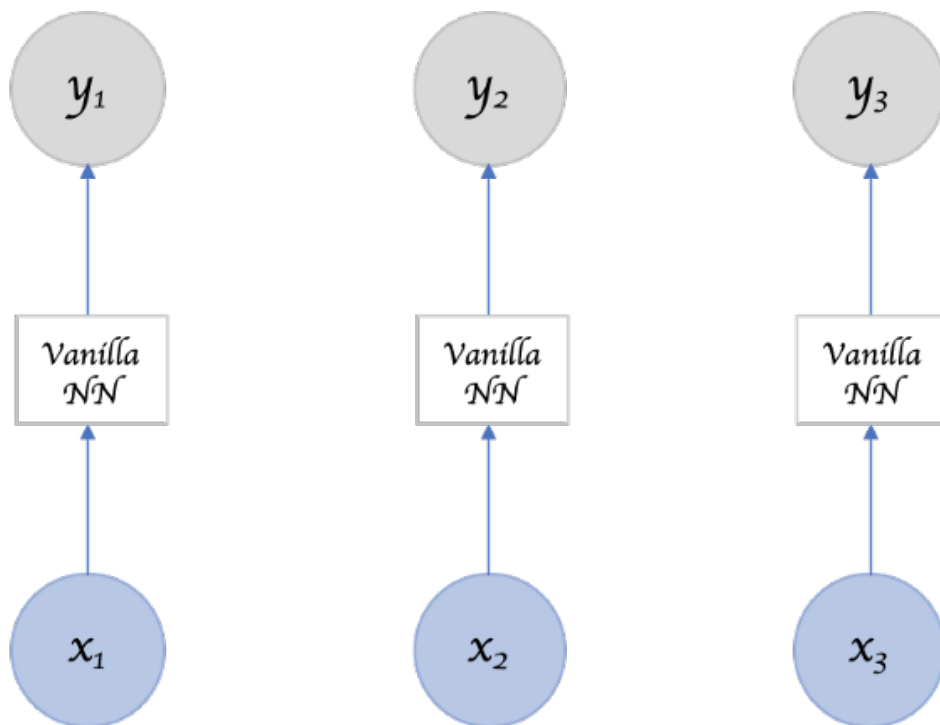
(a) 全连接层



(b) 卷积层

前馈网络 (Forward Feed Neural Network)

- 假设每次输入都是独立的，也就是说每次网络的输出只依赖于当前的输入。



如何给网络增加记忆能力？

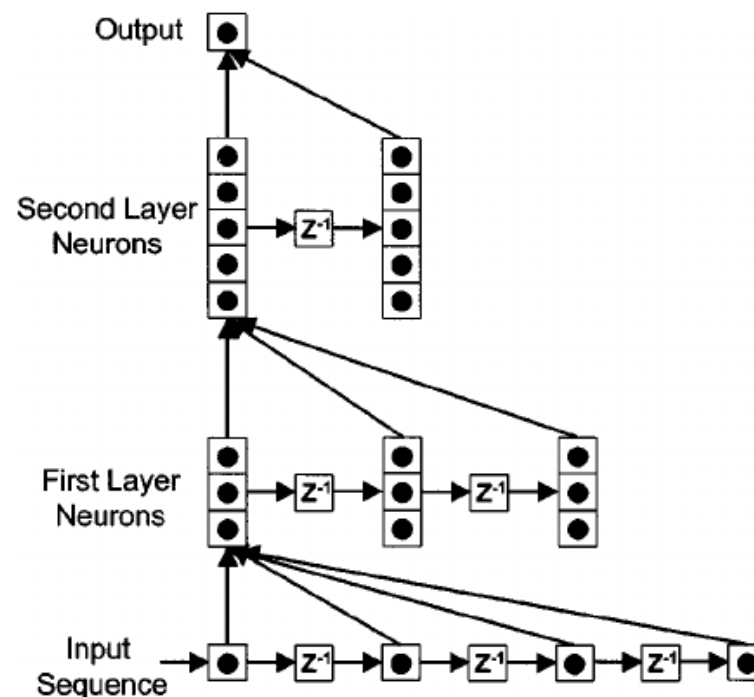
▶ 延时神经网络 (Time Delay Neural Network, TDNN)

- ▶ 建立一个额外的延时单元，用来存储网络的历史信息（可以包括输入、输出、隐状态等）

$$\mathbf{h}_t^{(l)} = f(\mathbf{h}_t^{(l-1)}, \mathbf{h}_{t-1}^{(l-1)}, \dots, \mathbf{h}_{t-K}^{(l-1)})$$

- ▶ 这样，前馈网络就具有了短期记忆的能力。

好像你记笔记时翻前几页回顾，记住的是前面几步。



<https://www.researchgate.net/publication/12314435> Neural system identification model of human sound localization

如何给网络增加记忆能力？

▶ 自回归模型 (Autoregressive Model, AR)

- ▶ 一类时间序列模型，用变量 y_t 的历史信息来预测自己

$$y_t = w_0 + \sum_{k=1}^K w_k y_{t-k} + \epsilon_t$$

- ▶ $\epsilon_t \sim N(0, \sigma^2)$ 为第 t 个时刻的噪声

▶ 有外部输入的非线性自回归模型 (Nonlinear Autoregressive with Exogenous Inputs Model, NARX)

$$y_t = f(\mathbf{x}_t, \mathbf{x}_{t-1}, \dots, \mathbf{x}_{t-K_x}, y_{t-1}, y_{t-2}, \dots, y_{t-K_y})$$

- ▶ 其中 $f(\cdot)$ 表示非线性函数，可以是一个前馈网络， K_x 和 K_y 为超参数。

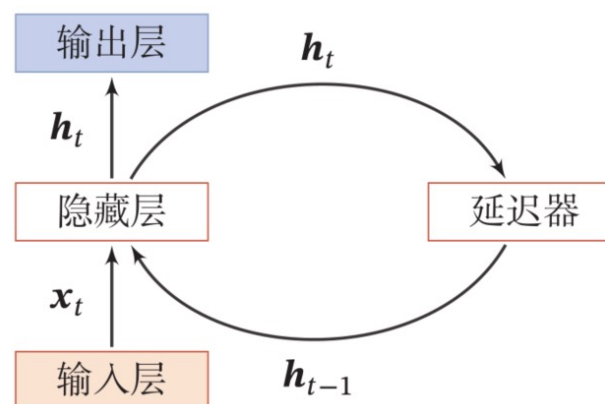
好像你总结了一张表，每次直接从表里查找过去的值来预测。

循环神经网络 (Recurrent Neural Network , RNN)

- ▶ 循环神经网络通过使用带自反馈的神经元，能够处理任意长度的时序数据。

$$\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t)$$

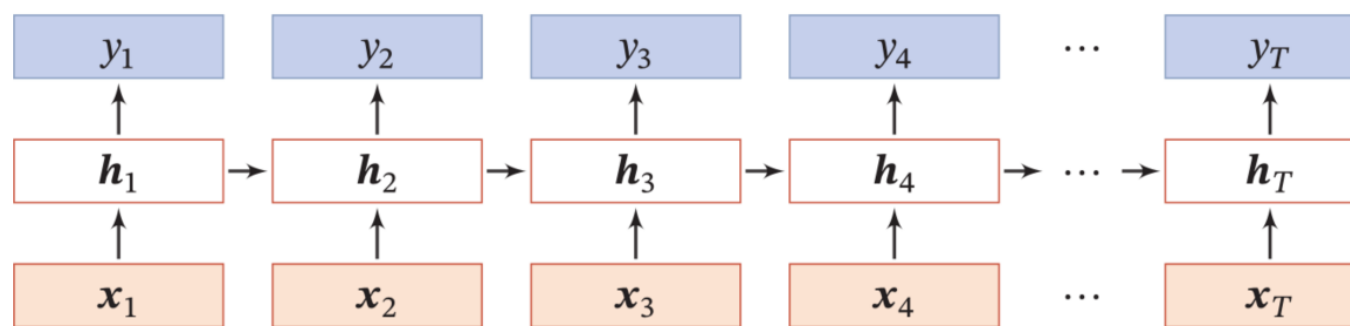
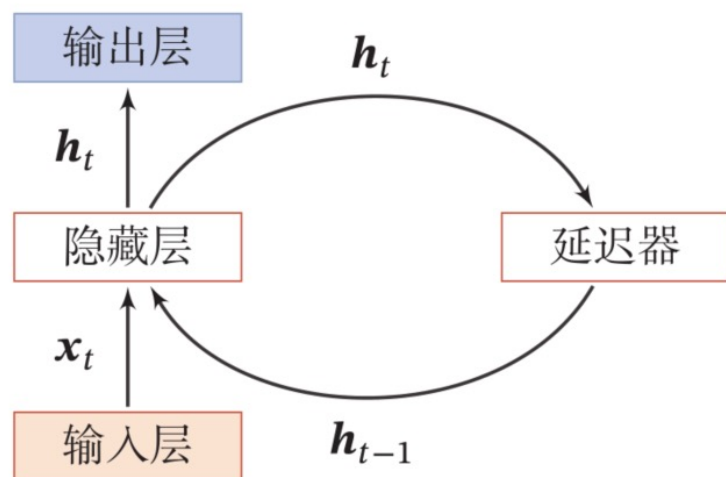
活性值
状态



- ▶ 循环神经网络比前馈神经网络更加符合生物神经网络的结构。
- ▶ 循环神经网络已经被广泛应用于语音识别、语言模型以及自然语言生成等任务上

好像你脑海中有一个状态，它随着每次经历而更新记忆。

按时间展开



简单循环网络 (Simple Recurrent Network , SRN)

► 状态更新:

$$\mathbf{h}_t = f(\mathbf{U}\mathbf{h}_{t-1} + \mathbf{W}\mathbf{x}_t + \mathbf{b})$$

► 一个完全连接的循环网络是任何非线性动力系统的近似器。

定理 6.1 – 循环神经网络的通用近似定理 [Haykin, 2009]: 如果一个完全连接的循环神经网络有足够数量的 sigmoid 型隐藏神经元, 它可以以任意的准确率去近似任何一个非线性动力系统

$$\mathbf{s}_t = g(\mathbf{s}_{t-1}, \mathbf{x}_t), \quad (6.10)$$

$$\mathbf{y}_t = o(\mathbf{s}_t), \quad (6.11)$$

其中 $\mathbf{s}_t$ 为每个时刻的隐状态, $\mathbf{x}_t$ 是外部输入, $g(\cdot)$ 是可测的状态转换函数, $o(\cdot)$ 是连续输出函数, 并且对状态空间的紧致性没有限制.

图灵完备

- ▶ 图灵完备 (Turing Completeness) 是指一种数据操作规则，比如一种计算机编程语言，可以实现图灵机的所有功能，解决所有的可计算问题。

定理 6.2 – 图灵完备 [Siegelmann et al., 1991]: 所有的图灵机都可以被一个由使用 Sigmoid 型激活函数的神经元构成的全连接循环网络来进行模拟。

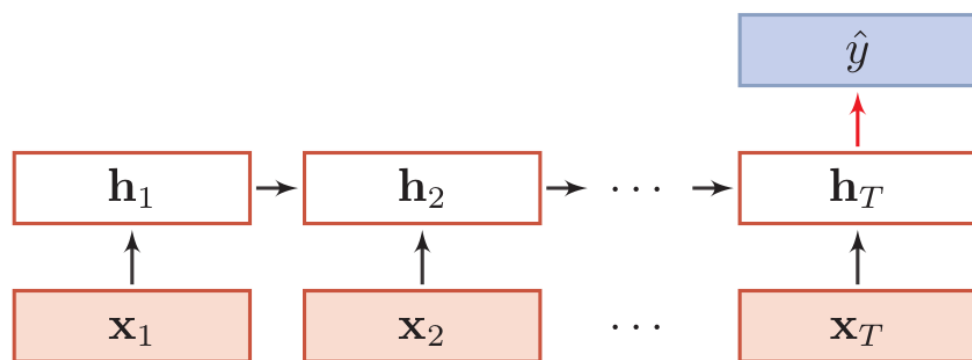
- ▶ 一个完全连接的循环神经网络可以近似解决所有的可计算问题。

应用到机器学习

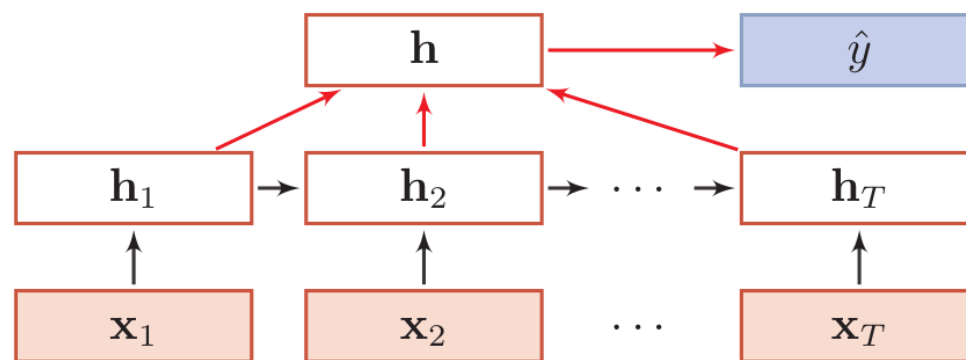
应用到机器学习

- ▶ 序列到类别
- ▶ 同步的序列到序列模式
- ▶ 异步的序列到序列模式

► 序列到类别



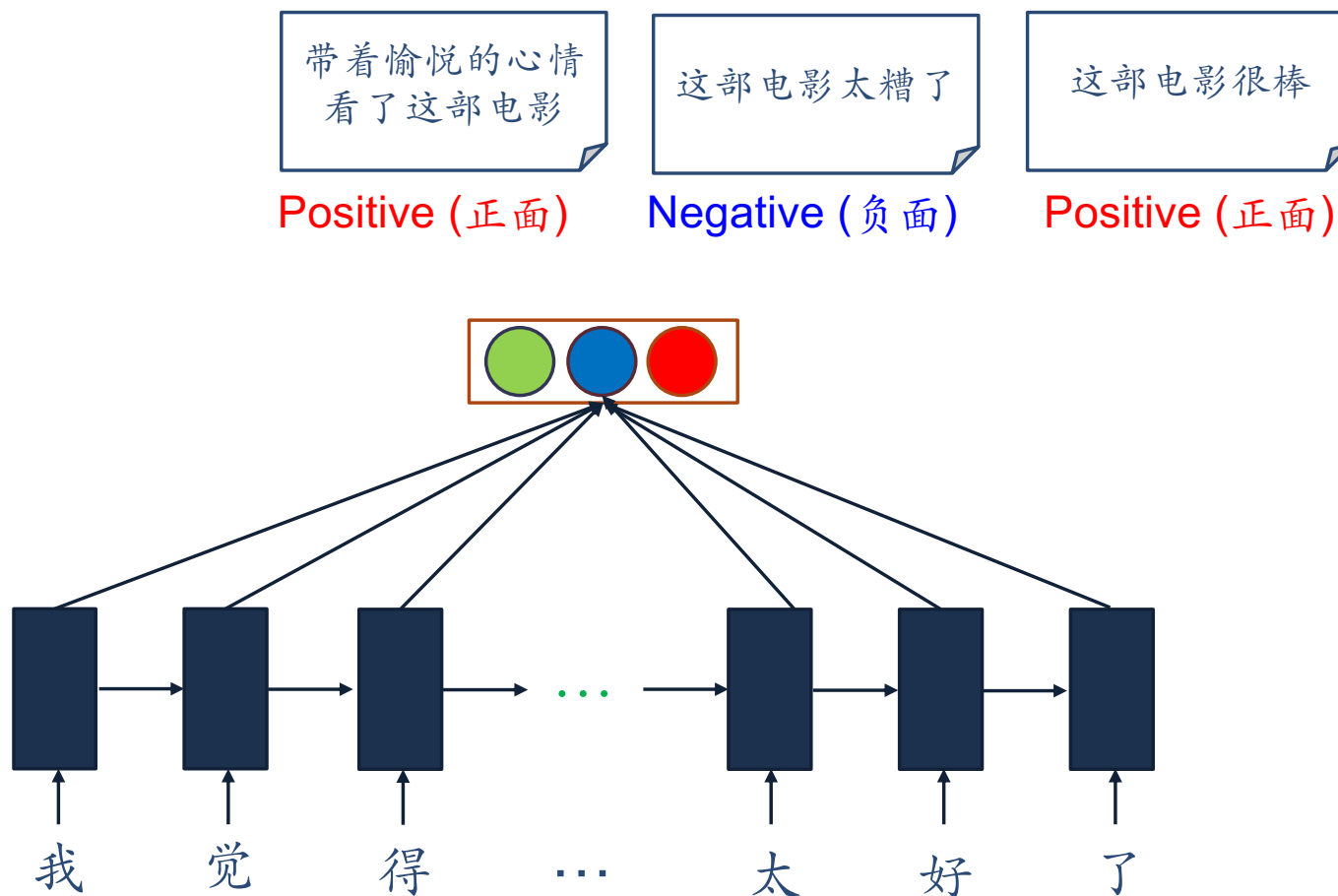
(a) 正常模式



(b) 按时间进行平均采样模式

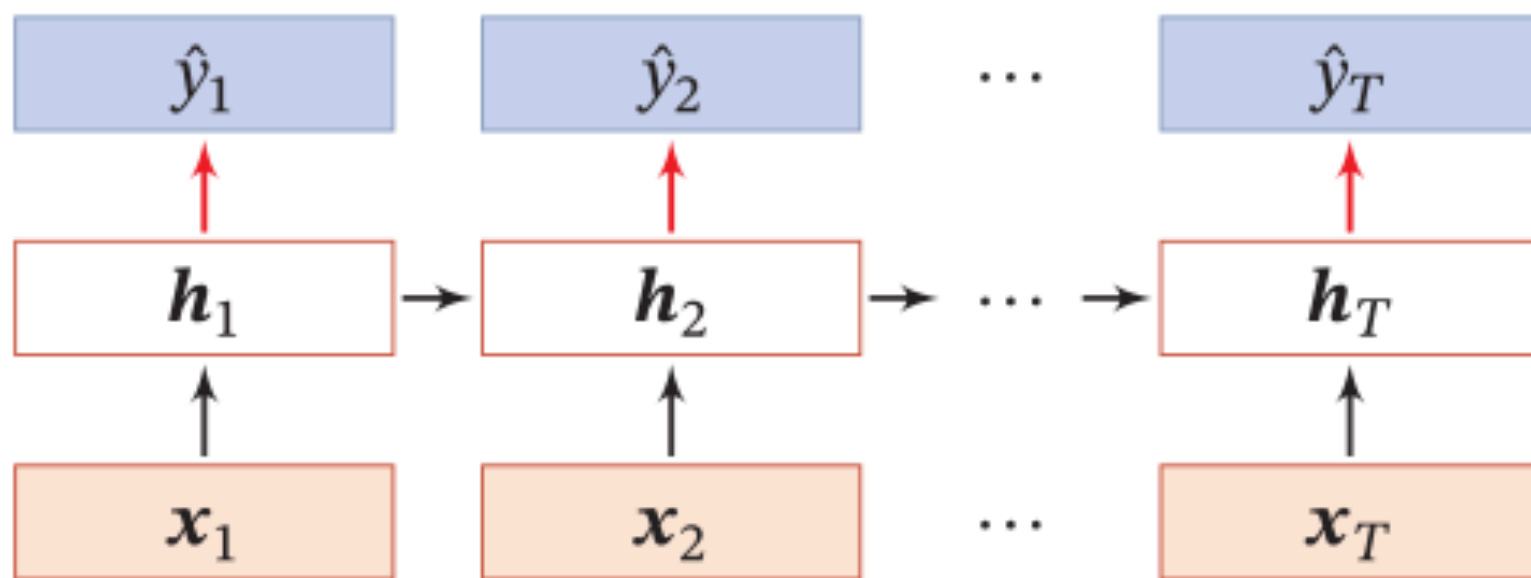
序列到类别

► 情感分类



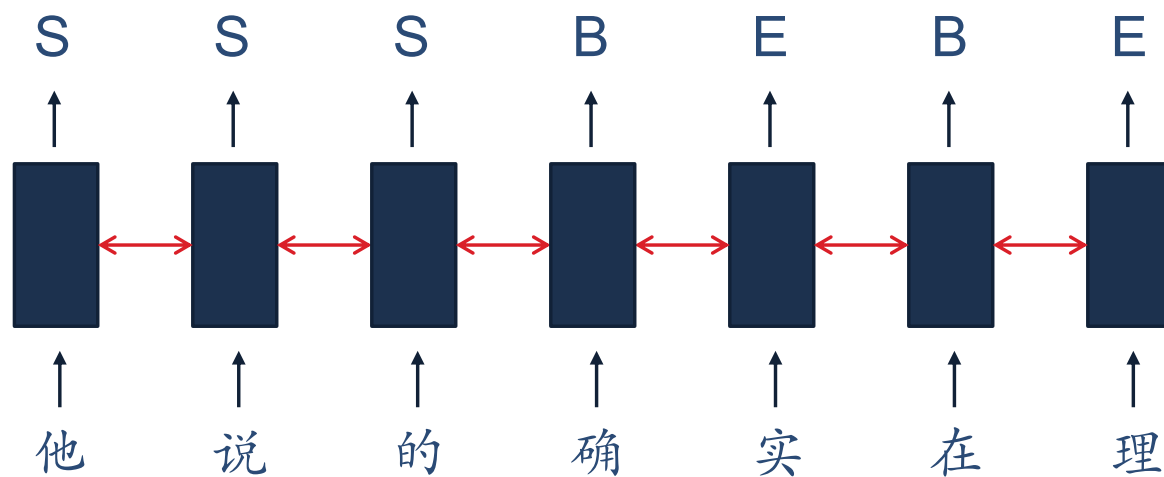
应用到机器学习

► 同步的序列到序列模式

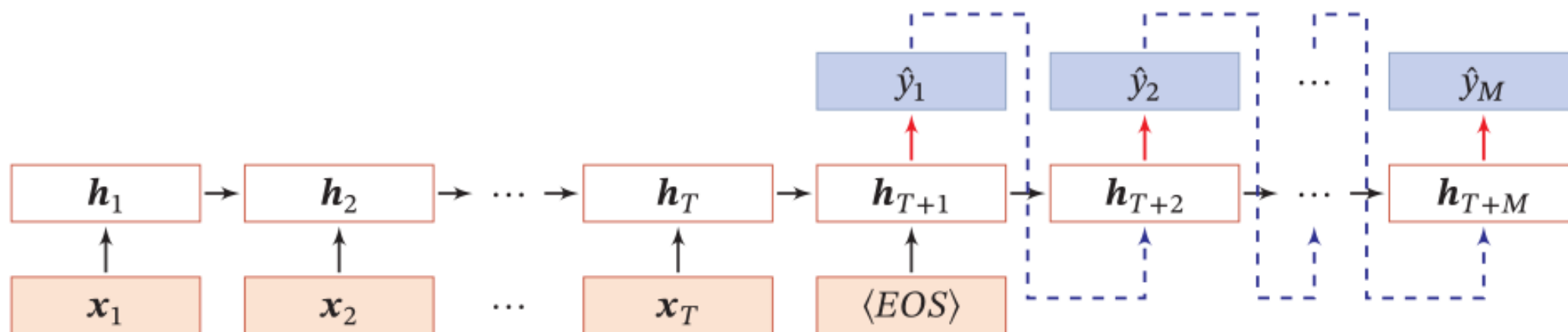


同步的序列到序列模式

► 中文分词

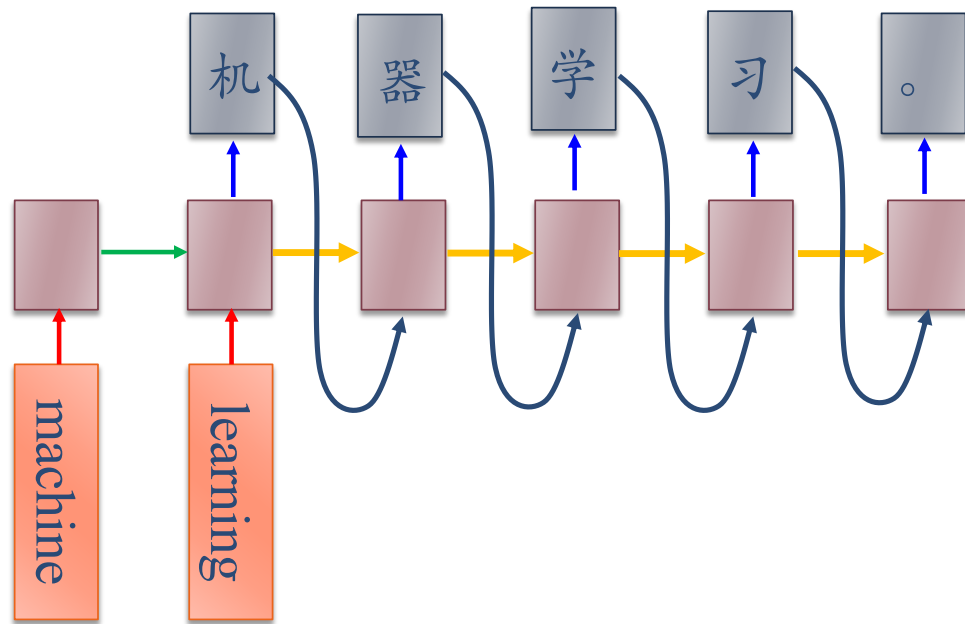


► 异步的序列到序列模式



异步的序列到序列模式

► 机器翻译



参数学习

▶ 机器学习

▶ 给定一个训练样本 $(\mathbf{x}, \mathbf{y})$ ，其中

▶ $\mathbf{x} = (x_1, \dots, x_T)$ 为长度是 T 的输入序列，

▶ $\mathbf{y} = (y_1, \dots, y_T)$ 是长度为 T 的标签序列。

▶ 时刻 t 的瞬时损失函数为

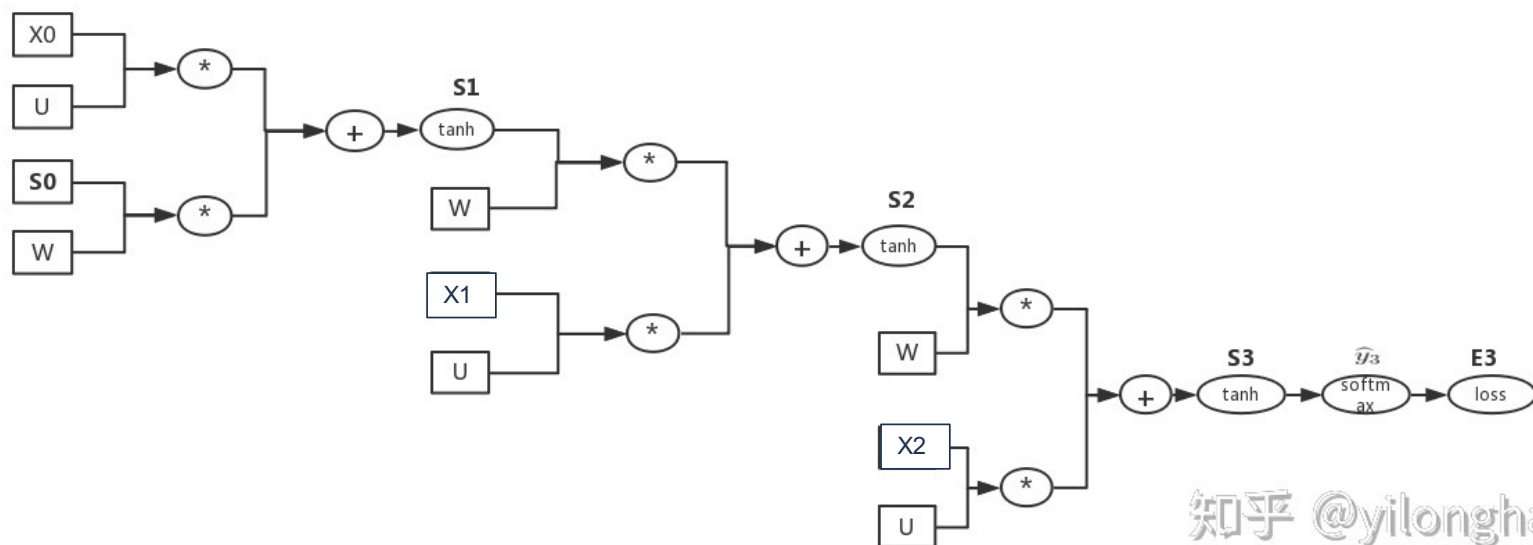
$$\mathcal{L}_t = \mathcal{L}(\mathbf{y}_t, g(\mathbf{h}_t)),$$

▶ 总损失函数

$$\mathcal{L} = \sum_{t=1}^T \mathcal{L}_t.$$

梯度

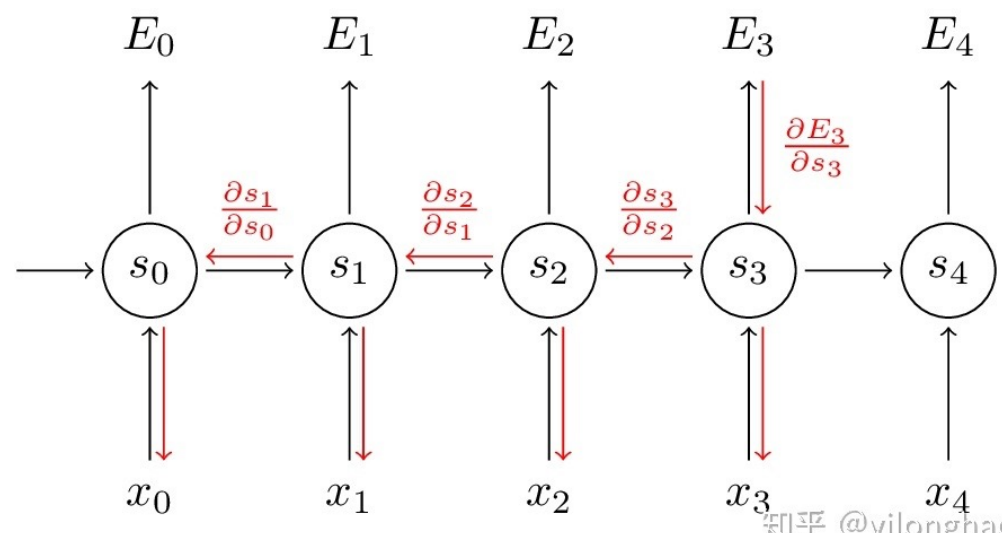
▶ 随时间反向传播算法 (Backpropagation Through Time, BPTT)



$$\frac{\partial E_3}{\partial W} = \frac{\partial E_3}{\partial \hat{y}_3} \frac{\partial \hat{y}_3}{\partial s_3} \frac{\partial s_3}{\partial W} + \frac{\partial E_3}{\partial \hat{y}_3} \frac{\partial \hat{y}_3}{\partial s_3} \frac{\partial s_3}{\partial s_2} \frac{\partial s_2}{\partial W} + \frac{\partial E_3}{\partial \hat{y}_3} \frac{\partial \hat{y}_3}{\partial s_3} \frac{\partial s_3}{\partial s_2} \frac{\partial s_2}{\partial s_1} \frac{\partial s_1}{\partial W}$$
$$\frac{\partial E_3}{\partial U} = \frac{\partial E_3}{\partial \hat{y}_3} \frac{\partial \hat{y}_3}{\partial s_3} \frac{\partial s_3}{\partial U} + \frac{\partial E_3}{\partial \hat{y}_3} \frac{\partial \hat{y}_3}{\partial s_3} \frac{\partial s_3}{\partial s_2} \frac{\partial s_2}{\partial U} + \frac{\partial E_3}{\partial \hat{y}_3} \frac{\partial \hat{y}_3}{\partial s_3} \frac{\partial s_3}{\partial s_2} \frac{\partial s_2}{\partial s_1} \frac{\partial s_1}{\partial U}$$

梯度

▶ 随时间反向传播算法 (Backpropagation Through Time, BPTT)

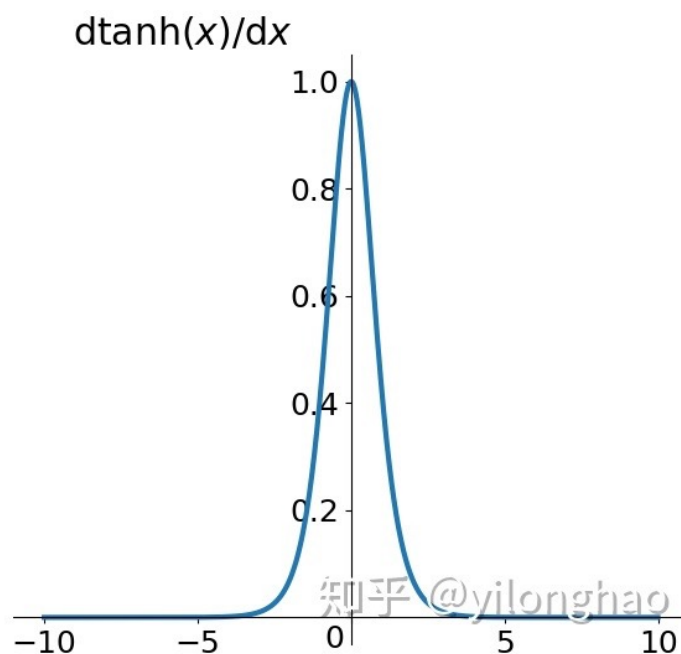
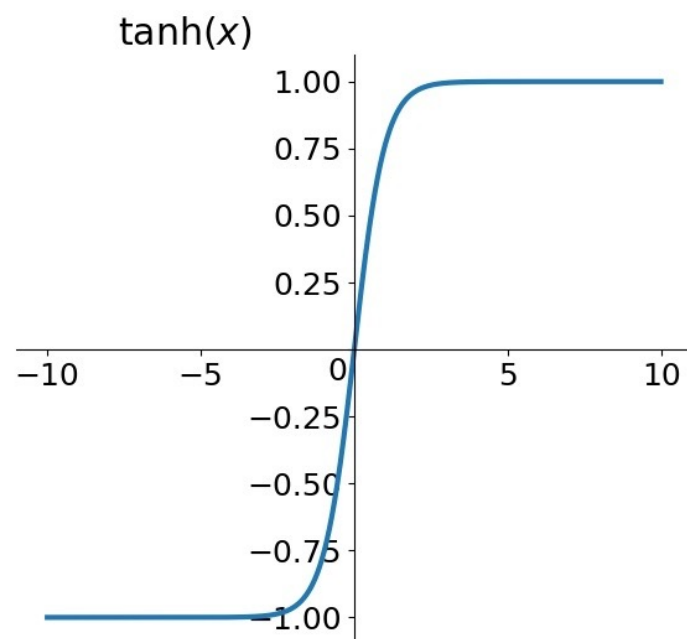


$$\frac{\partial E_3}{\partial W} = \sum_{k=0}^3 \frac{\partial E_3}{\partial \hat{y}_3} \frac{\partial \hat{y}_3}{\partial s_3} \left(\prod_{j=k+1}^3 \frac{\partial s_j}{\partial s_{j-1}} \right) \frac{\partial s_k}{\partial W}$$

梯度消失/爆炸

使用的激活函数是tanh

RNN梯度消失和CNN梯度消失不一样



$$s_3 = \tanh(Ux_t + Ws_2)$$

$$\frac{\partial s_3}{\partial s_2} = \tanh'(Ux_t + Ws_2)W$$

连乘，梯度消失或者爆炸

由于梯度爆炸(exploding)或消失(vanishing gradient)问题，实际上只能学习到短周期的依赖关系。这就是所谓的长程依赖问题。

长程依赖问题

► 循环神经网络在时间维度上非常深！

► 梯度消失或梯度爆炸

► 如何改进？

► 梯度爆炸问题

► 权重衰减

► 梯度截断

► 梯度消失问题

► 改进模型

IEEE TRANSACTIONS ON NEURAL NETWORKS, VOL. 5, NO. 2, MARCH 1994

157

Learning Long-Term Dependencies with Gradient Descent is Difficult

Yoshua Bengio, Patrice Simard, and Paolo Frasconi, *Student Member, IEEE*

Abstract—Recurrent neural networks can be used to map input sequences to output sequences, such as for recognition, production or prediction problems. However, practical difficulties have been reported in training recurrent neural networks to perform tasks in which the temporal contingencies present in the input/output sequences span long intervals. We show why gradient based learning algorithms face an increasingly difficult problem as the duration of the dependencies to be captured increases. These results expose a trade-off between efficient learning by gradient descent and latching on information for long periods. Based on an understanding of this problem, alternatives to standard gradient descent are considered.

I. INTRODUCTION

WE ARE INTERESTED IN training recurrent neural networks to map input sequences to output sequences, for applications in sequence recognition, production, or time-series prediction. All of the above applications require a system that will store and update context information; i.e., information computed from the past inputs and useful to produce desired outputs. Recurrent neural networks are well suited for those tasks because they have an internal state that can represent context information. The cycles in the graph of a recurrent network allow it to keep information about past inputs for an

a fully connected recurrent network) but are local in time; i.e., they can be applied in an on-line fashion, producing a partial gradient after each time step. Another algorithm was proposed [10], [18] for training constrained recurrent networks in which dynamic neurons—with a single feedback to themselves—have only incoming connections from the input layer. It is local in time like the forward propagation algorithms and it requires computation only proportional to the number of weights, like the back-propagation through time algorithm. Unfortunately, the networks it can deal with have limited storage capabilities for dealing with general sequences [7], thus limiting their representational power.

A task displays long-term dependencies if prediction of the desired output at time t depends on input presented at an earlier time $\tau \ll t$. Although recurrent networks can in many instances outperform static networks [4], they appear more difficult to train optimally. Earlier experiments indicated that their parameters settle in sub-optimal solutions that take into account short-term dependencies but not long-term dependencies [5]. Similar results were obtained by Mozer [19]. It was found that back-propagation was not sufficiently powerful to discover contingencies spanning long temporal

长短期记忆神经网络 (Long Short-Term Memory, LSTM)

将Tanh换成ReLU激活函数呢?

arxiv: [1504.00941] A Simple Way to
Initialize Recurrent Networks of Rectified
Linear Units



Recent research on deep feedforward networks has also produced some impressive results [19, 3] and there is now a consensus that for deep networks, rectified linear units (ReLUs) are easier to train than the logistic or tanh units that were used for many years [27, 40]. At first sight, ReLUs seem inappropriate for RNNs because they can have very large outputs so they might be expected to be far more likely to explode than units that have bounded values. A second aim of this paper is to explore whether ReLUs can be made to work well in RNNs and whether the ease of optimizing them in feedforward nets transfers to RNNs.

RNN中直接把激活函数换成ReLU会导致非常大的输出值

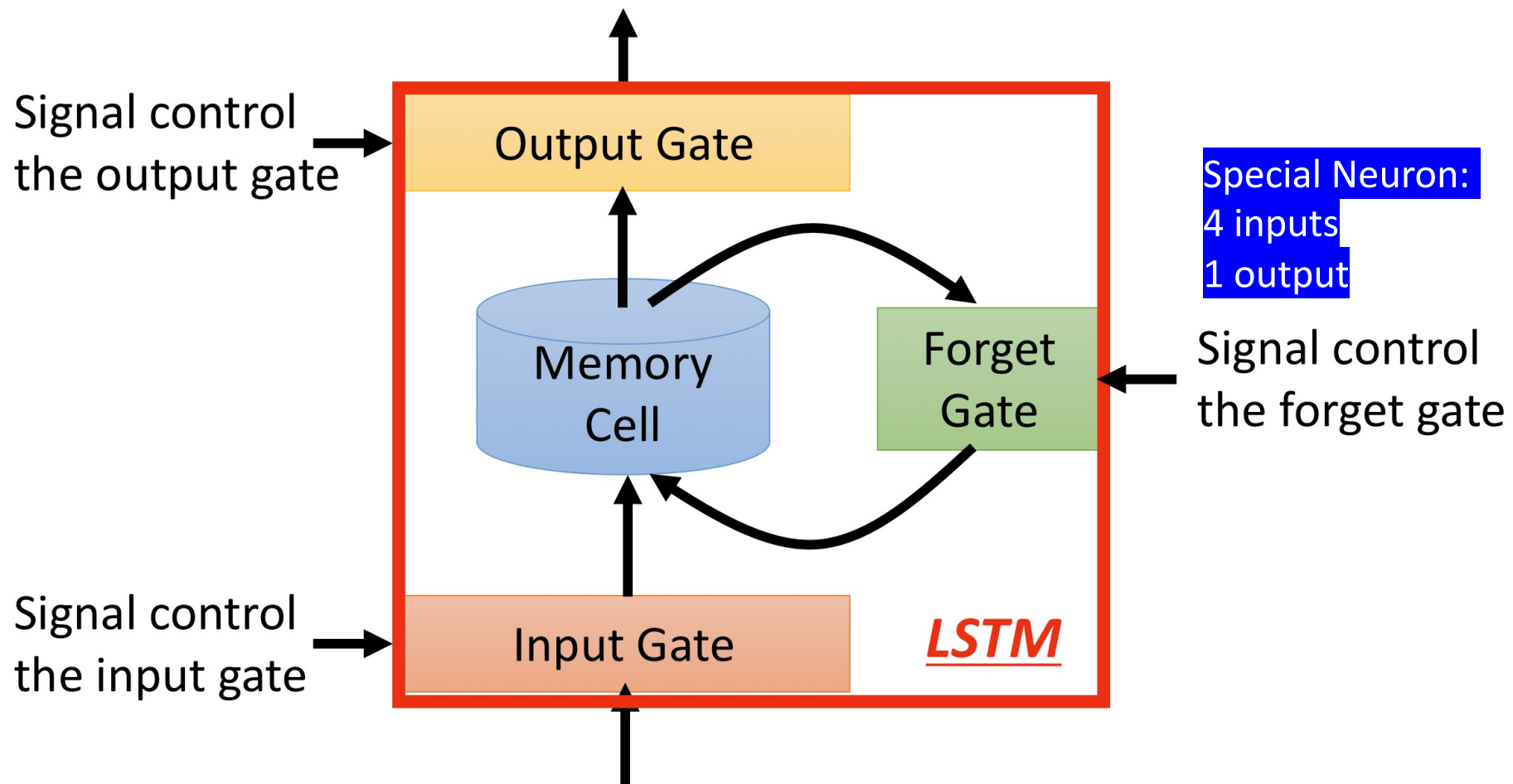
A Simple Way to Initialize Recurrent Networks of Rectified Linear Units

Quoc V. Le, Navdeep Jaitly, Geoffrey E. Hinton
Google

Abstract

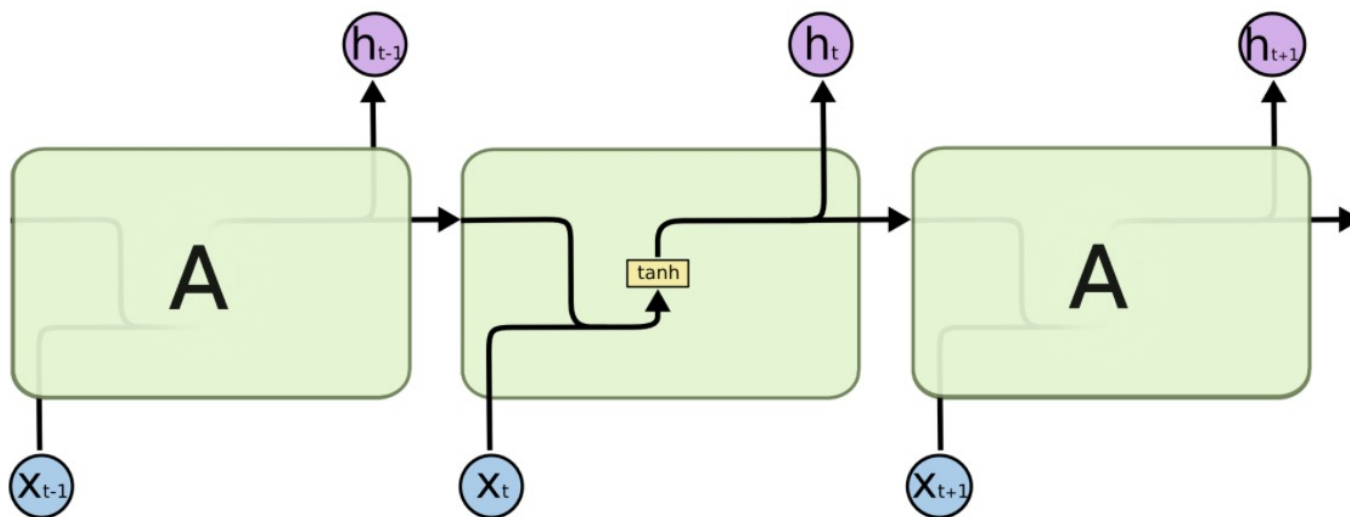
Learning long term dependencies in recurrent networks is difficult due to vanishing and exploding gradients. To overcome this difficulty, researchers have developed sophisticated optimization techniques and network architectures. In this paper, we propose a simpler solution that use recurrent neural networks composed of rectified linear units. Key to our solution is the use of the identity matrix or its scaled version to initialize the recurrent weight matrix. We find that our solution is comparable to a standard implementation of LSTMs on our four benchmarks: two toy problems involving long-range temporal structures, a large language modeling problem and a benchmark speech recognition problem.

长短期记忆神经网络 (Long Short-Term Memory, LSTM)

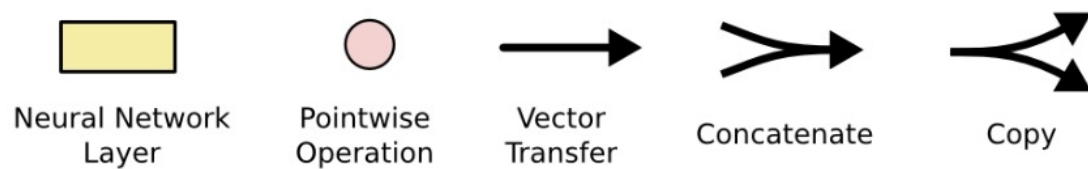


长短期记忆神经网络 (Long Short-Term Memory, LSTM)

LSTM 属于一种特殊的RNN, Hochreiter & Schmidhuber (1997)

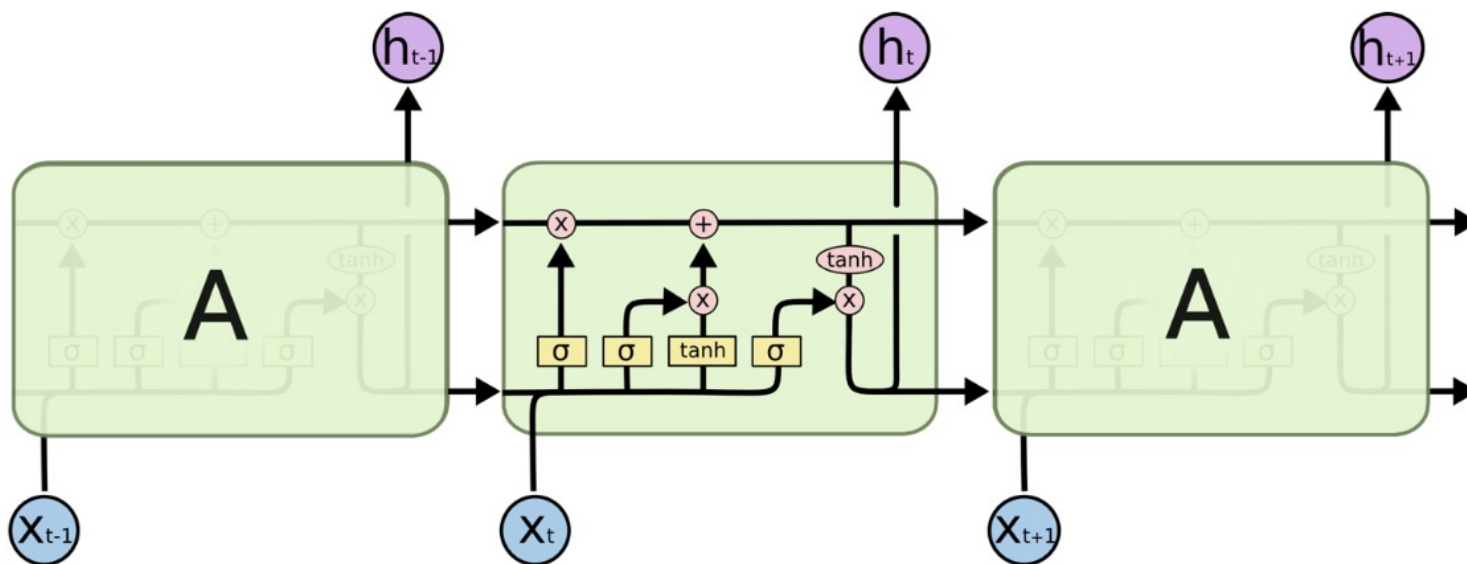


The repeating module in a standard RNN contains a single layer.



长短期记忆神经网络 (Long Short-Term Memory, LSTM)

LSTM 属于一种特殊的RNN，Hochreiter & Schmidhuber (1997)

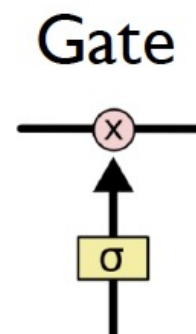
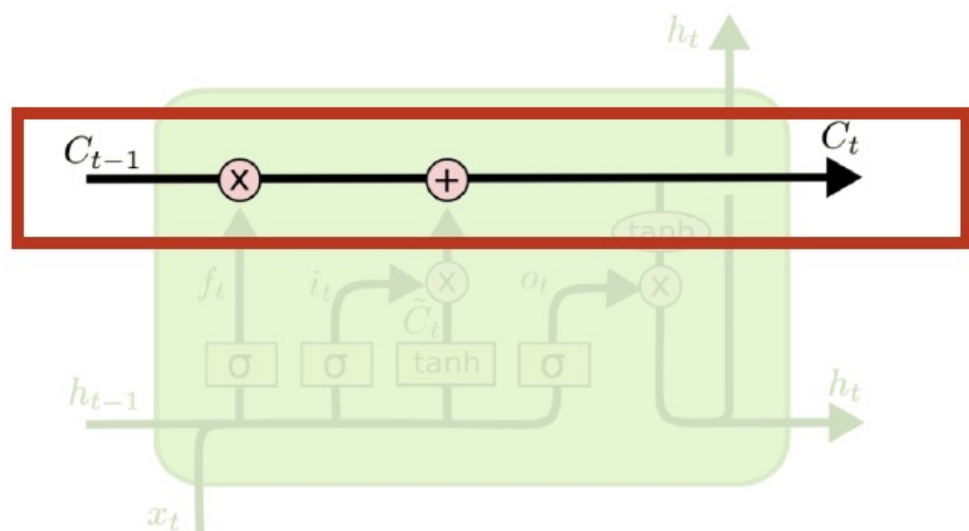


The repeating module in an LSTM contains four interacting layers.

长短期记忆神经网络 (Long Short-Term Memory, LSTM)

LSTM的核心思想

关键部分: cell state



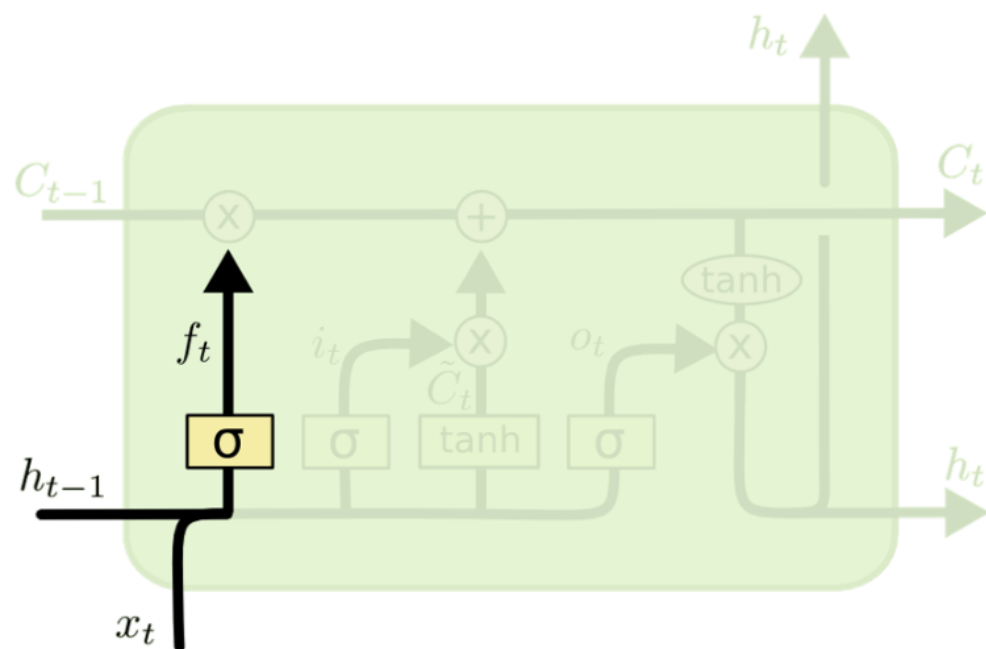
0: “let nothing through”
1: “let everything through”

LSTM用三种门来控制cell state

长短期记忆神经网络 (Long Short-Term Memory, LSTM)

Forget gate

Forget门用于控制哪些信息该遗忘？ 哪些信息该记忆？



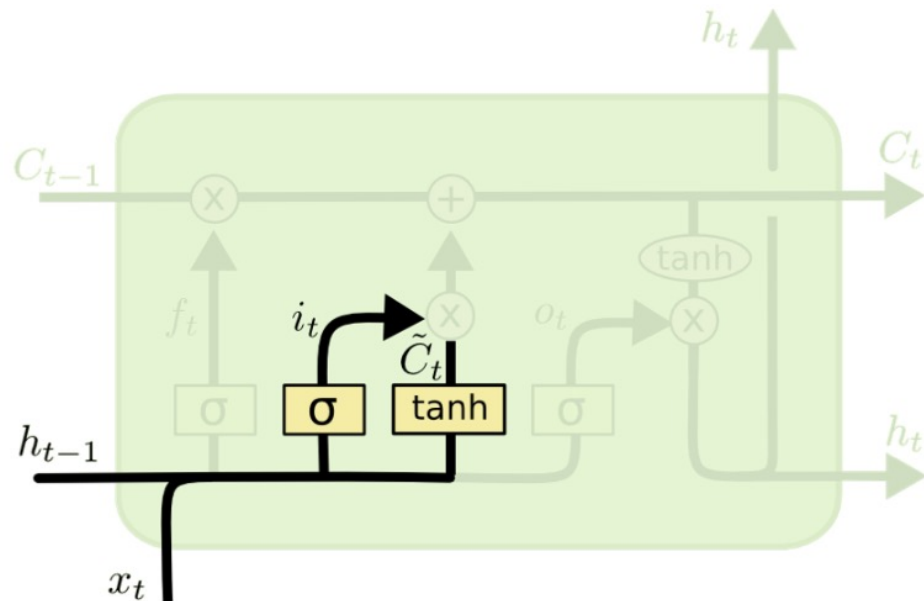
$$f_t = \sigma (W_f \cdot [h_{t-1}, x_t] + b_f)$$

长短期记忆神经网络 (Long Short-Term Memory, LSTM)

Input gate 门包括两部分

A perceptron layer: 确定输入中是否有感兴趣的地方?

A gate: 确定是否进行记忆?



$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

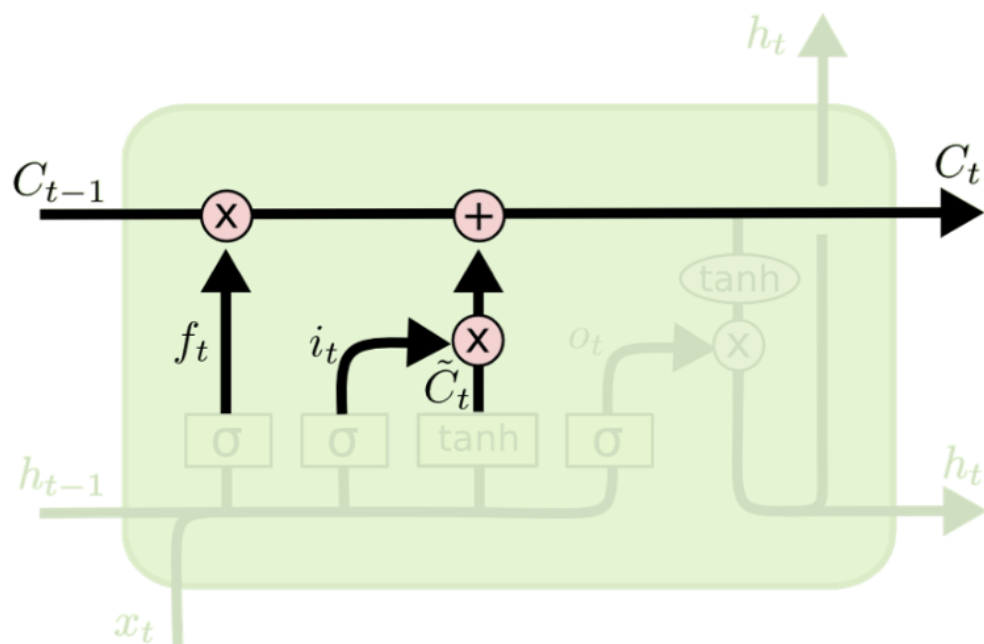
长短期记忆神经网络 (Long Short-Term Memory, LSTM)

Input gate 门包括两部分

A perceptron layer: 确定输入中是否有感兴趣的地方?

A gate: 确定是否进行记忆?

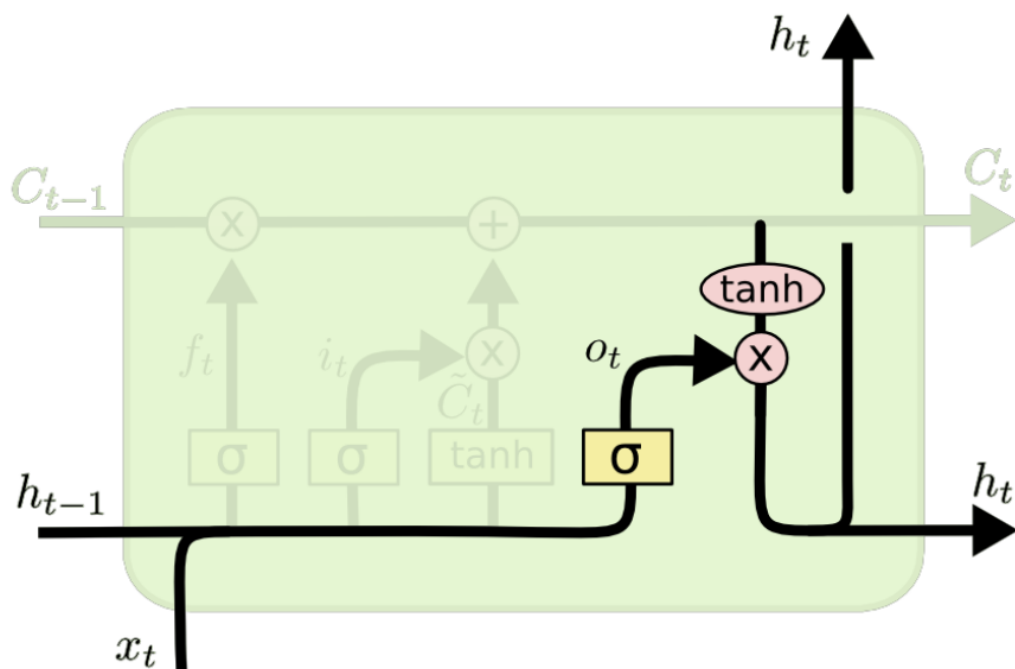
如果需要记忆, 加入cell state



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

长短期记忆神经网络 (Long Short-Term Memory, LSTM)

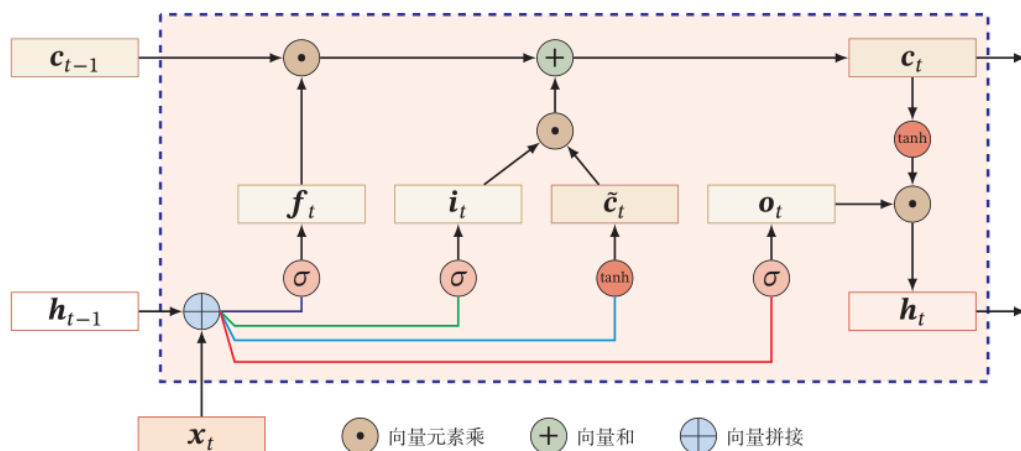
Output gate 用输入和记忆内容计算神经元输出



$$o_t = \sigma(W_o [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh(C_t)$$

长短期记忆神经网络 (Long Short-Term Memory, LSTM)



$$\mathbf{i}_t = \sigma(W_i \mathbf{x}_t + U_i \mathbf{h}_{t-1} + \mathbf{b}_i),$$

$$\mathbf{f}_t = \sigma(W_f \mathbf{x}_t + U_f \mathbf{h}_{t-1} + \mathbf{b}_f),$$

$$\mathbf{o}_t = \sigma(W_o \mathbf{x}_t + U_o \mathbf{h}_{t-1} + \mathbf{b}_o),$$

$$\tilde{\mathbf{c}}_t = \tanh(W_c \mathbf{x}_t + U_c \mathbf{h}_{t-1} + \mathbf{b}_c)$$

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t,$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t),$$

长短期记忆神经网络 (Long Short-Term Memory, LSTM)

LSTM为何可以缓解（而非消除）梯度消失问题？

$$\text{RNN} \quad \frac{\partial h_t}{\partial h_{t-1}} = (1 - h_t^2)W_h$$

$$\begin{aligned} \text{LSTM} \quad \frac{\partial C_t}{\partial C_{t-1}} = & C_{t-1} \sigma'(\cdot) W_{fh} * O_{t-1} \tanh'(C_{t-1}) + \tilde{C}_t \sigma' W_{ih} * O_{t-1} \tanh'(C_{t-1}) + \\ & I_t \tanh'(\cdot) W_{ch} * O_{t-1} \tanh'(C_{t-1}) + F_t \end{aligned}$$

长短期记忆神经网络 (Long Short-Term Memory, LSTM)

Keras和Pytorch支持

如果不明白LSTMs讲了些什么，不要担心

- Keras和Pytorch帮你处理一切
LSTM, GRU, RNN_TANH, RNN_RELU



长短期记忆神经网络 (Long Short-Term Memory, LSTM)

LSTM的神经元个数

$$\begin{pmatrix} a_1^1 & a_1^2 & \cdots & a_1^5 \\ a_2^1 & a_2^2 & \cdots & a_2^5 \\ \vdots & \vdots & \ddots & \vdots \\ a_{13}^1 & a_{13}^2 & \cdots & a_{13}^5 \end{pmatrix}$$

Shape=(13,5),时间步是13,
每个时间步的特征长度是5

```
1. from keras.layers import LSTM
2. from keras.models import Sequential
3.
4. time_step=13
5. featrue=5
6. hiddenfeatrue=10
7.
8. model=Sequential()
9. model.add( LSTM(hiddenfeatrue,input_shape=(time_step,featrue)))
10. model.summary()
```

```
1.
2. Layer (type)                Output Shape                Param #
3. =====
4. lstm_8 (LSTM)                (None, 10)                  640
5. =====
6. Total params: 640
7. Trainable params: 640
8. Non-trainable params: 0
9.
10.
```

Why 640?

假设你一个时间步的特征长度是 n ,经过
该LSTM得到的长度是 m ,这样就可以算
出该LSTM层的神经元个数为:

$$Neurons_{all} = 4 \times ((n + m) \times m + m)$$

LSTM的各种变体

▶ 没有遗忘门

$$\mathbf{c}_t = \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t.$$

▶ 耦合输入门和遗忘门

$$\mathbf{f}_t + \mathbf{i}_t = \mathbf{1}.$$

▶ peephole连接

$$\mathbf{i}_t = \sigma(W_i \mathbf{x}_t + U_i \mathbf{h}_{t-1} + V_i \mathbf{c}_{t-1} + \mathbf{b}_i),$$

$$\mathbf{f}_t = \sigma(W_f \mathbf{x}_t + U_f \mathbf{h}_{t-1} + V_f \mathbf{c}_{t-1} + \mathbf{b}_f),$$

$$\mathbf{o}_t = \sigma(W_o \mathbf{x}_t + U_o \mathbf{h}_{t-1} + V_o \mathbf{c}_t + \mathbf{b}_o),$$

在标准LSTM中:

- 门控 (输入门 i_t 、遗忘门 f_t 、输出门 o_t) 只依赖当前输入 x_t 和前一时刻隐藏状态 h_{t-1} 来做决定。

而在带Peephole的LSTM中:

- 门控还额外观察 (窥视) 前一刻的Cell State c_{t-1} , 作为决策依据。

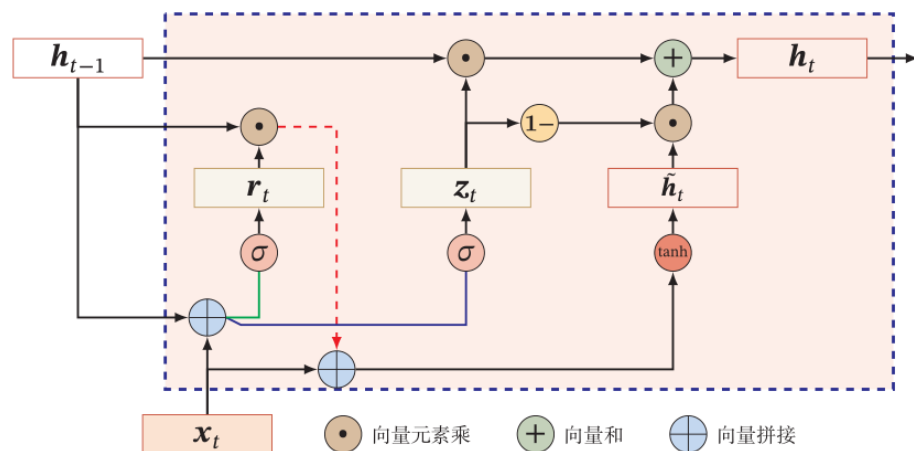
公式上, 门的计算变成:

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + V_f c_{t-1} + b_f)$$

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + V_i c_{t-1} + b_i)$$

$$o_t = \sigma(W_o x_t + U_o h_{t-1} + V_o c_t + b_o)$$

Gated Recurrent Unit, GRU 门控循环单元



重置门

$$\mathbf{r}_t = \sigma(\mathbf{W}_r \mathbf{x}_t + \mathbf{U}_r \mathbf{h}_{t-1} + \mathbf{b}_r),$$

$$\mathbf{z}_t = \sigma(\mathbf{W}_z \mathbf{x}_t + \mathbf{U}_z \mathbf{h}_{t-1} + \mathbf{b}_z),$$

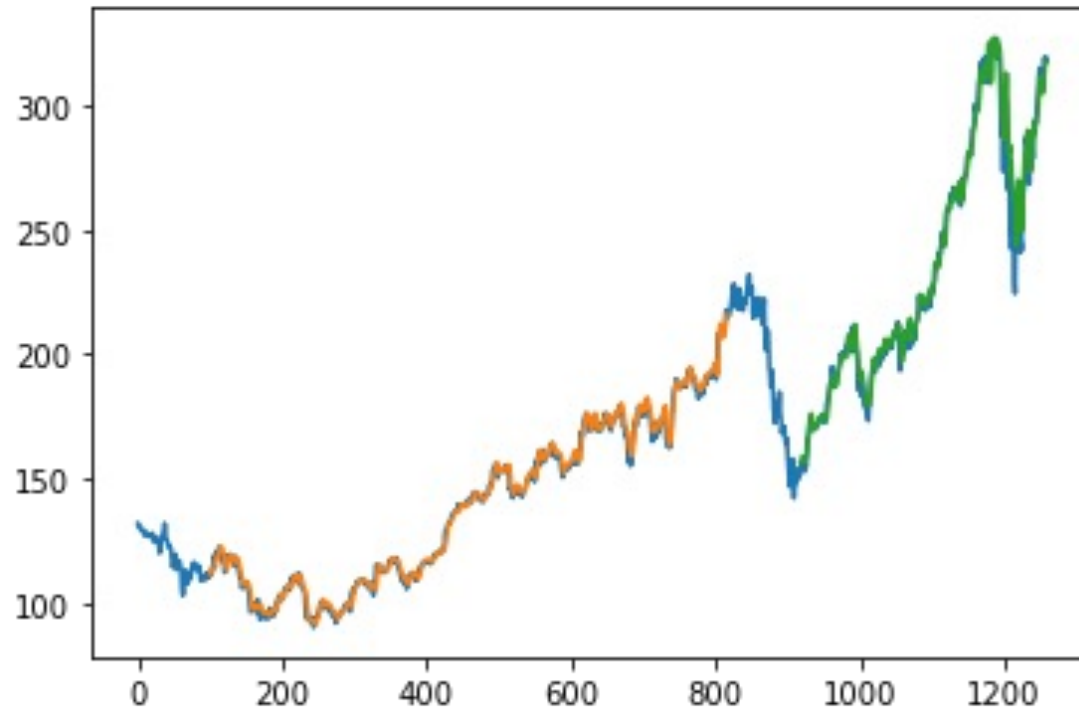
更新门

$$\tilde{\mathbf{h}}_t = \tanh(\mathbf{W}_c \mathbf{x}_t + \mathbf{U}(\mathbf{r}_t \odot \mathbf{h}_{t-1}))$$

$$\mathbf{h}_t = \mathbf{z}_t \odot \mathbf{h}_{t-1} + (1 - \mathbf{z}_t) \odot \tilde{\mathbf{h}}_t,$$

循环网络应用

股票走勢



- Green indicates the Predicted Data
- Blue indicates the Complete Data
- Orange indicates the Train Data

Stock Price Prediction and Forecasting using Stacked LSTM

<https://www.analyticsvidhya.com/blog/2021/05/stock-price-prediction-and-forecasting-using-stacked-lstm/>

天气预报



Star 53,247

About Keras

Getting started

Developer guides

Keras API reference

Code examples

Computer Vision

Natural Language Processing

Structured Data

Timeseries

Audio Data

Generative Deep Learning

Reinforcement Learning

Graph Data

Search Keras documentation...

» Code examples / Timeseries / Timeseries forecasting for weather prediction

Timeseries forecasting for weather prediction

Authors: Prabhanshu Attri, Yashika Sharma, Kristi Takach, Falak Shah

Date created: 2020/06/23

Last modified: 2020/07/20

Description: This notebook demonstrates how to do timeseries forecasting using a LSTM model.

[View in Colab](#) • [GitHub source](#)

Setup

This example requires TensorFlow 2.3 or higher.

```
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
```

Timeseries forecasting for weather prediction

» Setup

» Climate Data Time-Series

» Raw Data Visualization

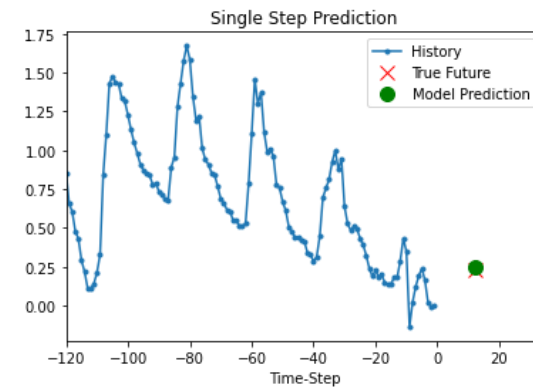
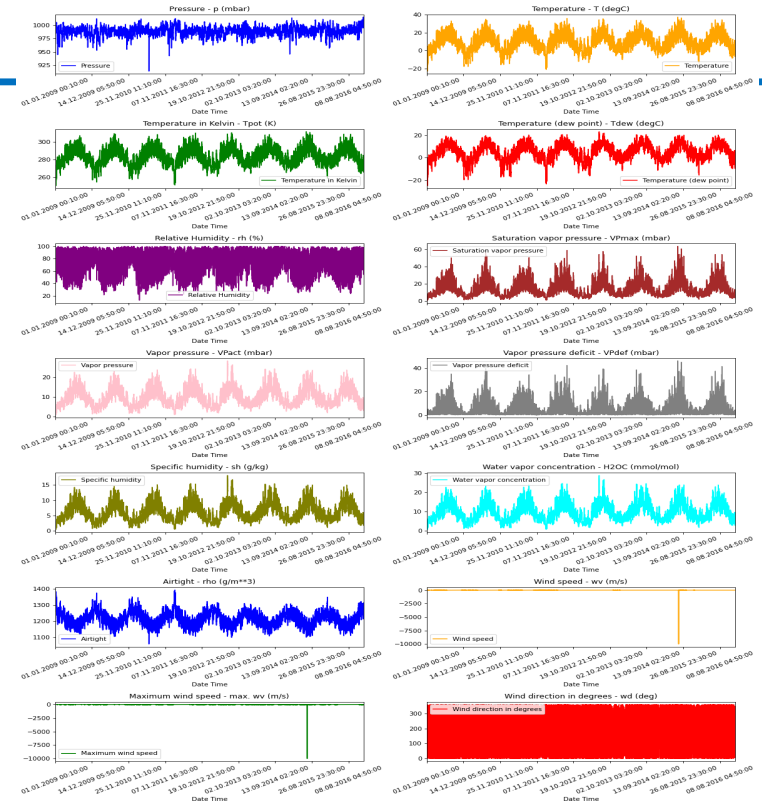
» Data Preprocessing

Training dataset

» Validation dataset

» Training

» Prediction



https://keras.io/examples/timeseries/timeseries_weather_forecasting/

循环神经网络总结

▶ 优点:

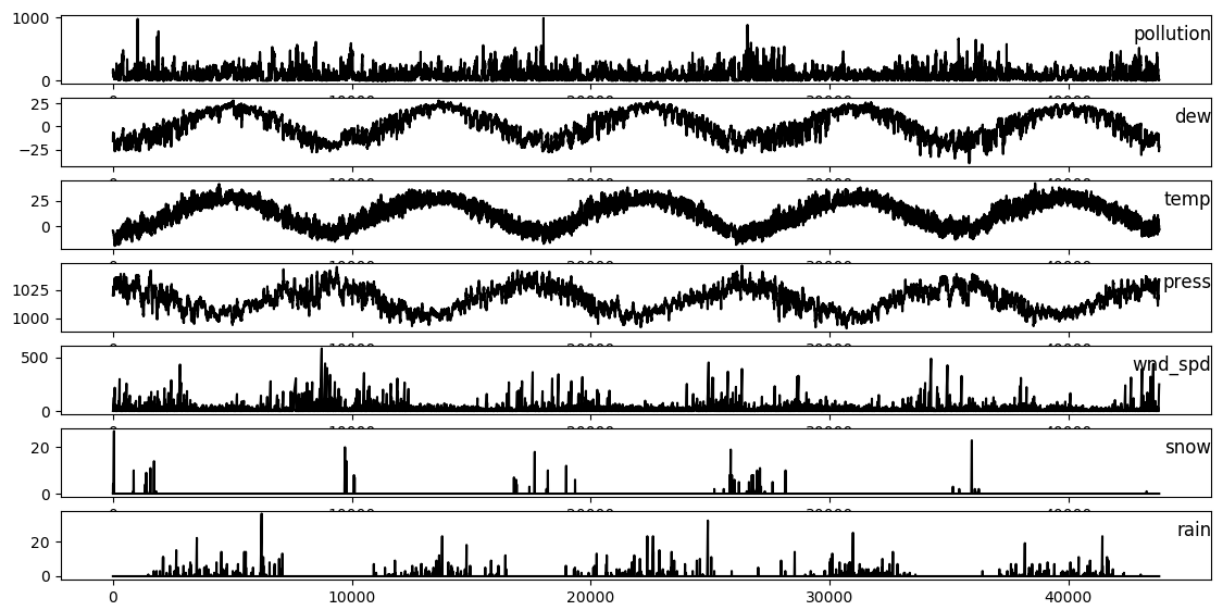
- ▶ 引入记忆
- ▶ 图灵完备

▶ 缺点:

- ▶ 长程依赖问题
- ▶ 记忆容量问题
- ▶ 并行能力

这些问题由在近年发展出的transformer，bert等方法，通过卷积网络增加attention机制解决了这些问题。

课后作业



PM 2.5 prediction

